# Praktikum Pemrograman Perangkat Bergerak



**Dosen Pengampu:**

**MOHAMMAD AS'AD ROSYADI**


**707012400139**

**Zahran Rauhillah Raziq**

**D4 SIKC 48-03**


## Pemrograman Perangkat Bergerak Lanjut

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan konsep Navigator, route, dan tumpukan navigasi (navigation stack) pada aplikasi Flutter.
- Membangun perpindahan halaman menggunakan Navigator.push() dan Navigator.pop() beserta MaterialPageRoute.
- Mengirim data ke halaman tujuan melalui konstruktor widget dan menerima data kembali melalui nilai kembalian Navigator.pop().
- Menerapkan named routes melalui properti initialRoute, routes, dan onGenerateRoute pada MaterialApp.
- Membedakan penggunaan Navigator.pushNamed(), pushReplacementNamed(), popUntil(), dan pushNamedAndRemoveUntil().
- Mengimplementasikan widget navigasi Material Design 3, yaitu NavigationBar (bottom navigation bar), BottomAppBar, Drawer atau NavigationDrawer, dan NavigationRail.
- Membuat tab navigation menggunakan DefaultTabController, TabBar, dan TabBarView.
- Menyusun navigasi adaptif yang berpindah otomatis antara NavigationBar dan NavigationRail berdasarkan lebar layar perangkat.
- Melakukan pengujian alur navigasi pada emulator, perangkat fisik, maupun peramban serta menelusuri galat yang berkaitan dengan konteks Navigator.

2. Alat dan Bahan

2.1 Perangkat Keras

- Laptop/PC: Laptop
- Processor: Intel Core i7
- RAM: 32 GB
- Penyimpanan: SSD dengan ruang kosong minimal 80 GB
- Sistem Operasi: Windows 11
- Perangkat pengujian: Emulator Android

2.2 Perangkat Lunak

- Flutter SDK: Versi 3.47.4
- Dart SDK: Versi 3.13.3
- Android Studio: Versi terbaru yang terpasang
- Visual Studio Code: Versi terbaru yang terpasang
- Git: Versi 2.55.0.windows.5
- Android Emulator: Perangkat virtual Android

3. Langkah Langkah Praktikum

**Soal 1: Menambahkan Manajemen State pada Aplikasi Layanan Warga "Nusantara Cerdas Mobile"**

a. Menambahkan Package Provider
Membuka terminal dan tambahkan package provider dengan perintah "flutter pub add provider" yang digunakan untuk mengelola dan membagikan data antara halaman lain tanpa harus meneruskan data secara manual melalui setiap widget

b. Membuat Model Favorit Layanan
Membuat file lib/models/favorit_model.dart. Model ini menggunakan ChangeNotifier untuk mengelola daftar layanan favorit dan memberi tahu widget Ketika terjadi perubahan

```dart
import 'package:flutter/foundation.dart';

class FavoritModel extends ChangeNotifier {
  final Set<String> _daftarFavorit = {};

  Set<String> get daftarFavorit => Set.unmodifiable(_daftarFavorit);

  bool isFavorit(String namaLayanan) {
    return _daftarFavorit.contains(namaLayanan);
  }

  void tandai(String namaLayanan) {
    if (_daftarFavorit.add(namaLayanan)) {
      notifyListeners();
    }
  }

  void batalTandai(String namaLayanan) {
    if (_daftarFavorit.remove(namaLayanan)) {
      notifyListeners();
    }
  }
}
```

Class FavoritModel menympan nama layanan yang dipilh pengguna. Method tandai() digunakan untuk menambahkan layanan, sedangkan batalTandai() digunakan untuk menghapusnya. Method notifyListeners() memberi tahu widget yang berlangganan bahwa data telah berubah.

c. Membuat Model Pengajuan Layanan
Membuat file lib/models/pengajuan_model.dart untuk menyimpan daftar layanan yang diajukan oleh pengguna.

```dart
import 'package:flutter/foundation.dart';

class PengajuanModel extends ChangeNotifier {
  final List<Map<String, String>> _daftarPengajuan = [];

  List<Map<String, String>> get daftarPengajuan =>
      List.unmodifiable(_daftarPengajuan);

  int get totalPengajuan => _daftarPengajuan.length;

  void tambahPengajuan(Map<String, String> layanan) {
    _daftarPengajuan.add(Map<String, String>.from(layanan));
    notifyListeners();
  }
}
```

Model ini memiliki method tambahPengajuan() untuk memasukkan layanan ke dalam daftar. Getter totalPengajuan digunakan untuk menghitung jumlah layanan yang telah diajukan. Setiap perubahan akan memicu pembaruan pada widget yang menggunakan model tersebut.

d. Menggabungkan Provider pada main.dart
Setelah kedua model selesai dibuat, lalu mengintegrasikannya ke dalam aplikasi melalu MultiProvider.

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'models/favorit_model.dart';
import 'models/pengajuan_model.dart';
import 'navigation/app_routes.dart';

Run | Debug | Profile
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => FavoritModel()),
        ChangeNotifierProvider(create: (_) => PengajuanModel()),
      ],
      child: const NusantaraCerdasNav(),
    ), // MultiProvider
  );
}
```

MultiProvider menyediakan kedua model untuk halaman-halaman di bawah widget utama aplikasi. Dengan demikian, halaman layanan, halaman warga, dan halaman detail dapat mengakses data favorit maupun pengajuan menggunakan Provider.

e. Menambahkan Fitur Favorit pada Halaman Layanan
Pada file lib/pages/halmaan_layanan.dart, setiap kartu layanan ditambahkan tombol ikon Bintang. Pengguna dapat menandai layanan sebagai favorit atau membatalkan pilihan tersebut.

```dart
// Tombol bintang favorit.
trailing: Row(
  mainAxisSize: MainAxisSize.min,
  children: [
    IconButton(
      tooltip: isFavorit
          ? 'Hapus dari favorit'
          : 'Tambahkan ke favorit',
      onPressed: () {
        final model = context.read<FavoritModel>();

        if (model.isFavorit(namaLayanan)) {
          model.batalTandai(namaLayanan);
        } else {
          model.tandai(namaLayanan);
        }
      },
      icon: Icon(
        isFavorit ? Icons.star : Icons.star_border,
        color: isFavorit ? Colors.amber : null,
      ), // Icon
    ), // IconButton
    const Icon(Icons.chevron_right),
  ],
), // Row
```

context.watch() digunakan untuk memantau perubahan data favorit, sedangkan context.read() digunakan untuk memanggil method pada model ketika tombol ditekan. Ikon bintang berwarna kuning menunjukkan bahwa layanan telah dipilih sebagai favorit.

f. Menampilkan Favorit dan Pengajuan pada Halaman Warga
Pada file lib/pages/halaman_warga.dart, digunakan widget Consumer untuk menampilkan data dari model favorit dan pengajuan.

```dart
Consumer<FavoritModel>(
  builder: (context, model, child) {
    final daftarFavorit = model.daftarFavorit;

    if (daftarFavorit.isEmpty) {
      return const Card(
        child: Padding(
          padding: EdgeInsets.all(20),
          child: Column(
            children: [
              Icon(Icons.star_border, size: 40),
              SizedBox(height: 8),
              Text('Belum ada layanan favorit.'),
              Text(
                'Tandai layanan dengan ikon bintang '
                'pada halaman Layanan.',
                textAlign: TextAlign.center,
              ), // Text
            ],
          ), // Column
        ), // Padding
      ); // Card
    }

    return Card(
      child: Column(
        children: daftarFavorit.map((nama) {
          return ListTile(
            leading: const Icon(Icons.star, color: Colors.amber),
            title: Text(nama),
            trailing: IconButton(
              tooltip: 'Hapus dari favorit',
              icon: const Icon(Icons.close),
              onPressed: () {
                context.read<FavoritModel>().batalTandai(nama);
              },
            ), // IconButton
          ); // ListTile
        }).toList(),
      ), // Column
    ); // Card
  },
), // Consumer
```

Widget Consumer<FavoritModel> membangun tampilan berdasarkan daftar favorit. Ketika pengguna menambahkan atau menghapus favorit dari halaman layanan, daftar pada halaman warga akan diperbarui secara otomatis.

Halaman warga juga menampilkan daftar pengajuan menggunakan Consumer<PengajuanModel> dan jumlah pengajuan melalui model.totalPengajuan. Dengan demikian, pengguna dapat melihat layanan yang telah diajukan tanpa harus membuka kembali halaman detail.

g. Menambahkan Stateful Widget dan Indikator Loading
Pada file lib/pages/halaman_detail_layanan.dart, halaman detail menggunakan StatefulWidget karena membutuhkan perubahan tampilan selama pengajuan berlangsung.

```dart
class _HalamanDetailLayananState extends State<HalamanDetailLayanan> {
  bool _sedangMengirim = false;

  Future<void> _ajukanPermohonan() async {
    if (_sedangMengirim) return;

    setState(() {
      _sedangMengirim = true;
    });

    // Simulasi proses pengiriman selama 1 detik.
    await Future.delayed(const Duration(seconds: 1));

    if (!mounted) return;

    // Simpan layanan ke model pengajuan.
    context.read<PengajuanModel>().tambahPengajuan(widget.dataLayanan);

    final namaLayanan = widget.dataLayanan['nama'] ?? 'Layanan';

    // Kembalikan hasil ke halaman Layanan.
    Navigator.pop(context, 'Permohonan $namaLayanan berhasil diajukan.');
  }
```

Variabel _sedangMengirim digunakan untuk mengontrol status loading. Method setState() memperbarui tampilan ketika proses pengajuan dimulai. Setelah jeda satu detik sebagai simulasi, layanan ditambahkan ke model pengajuan dan aplikasi kembali ke halaman sebelumnya menggunakan Navigator.pop().

Tombol pengajuan juga menampilkan CircularProgressIndicator selama proses berlangsung dan dinonaktifkan untuk mencegah penekanan berulang.

h. Menambahkan Badge pada Navigasi
Menampilkan jumlah pengajuan pada ikon Warga di file lib/navigation/kerangka_navigasi.dart. Badge digunakan sebagai indicator jumlah layanan yang telah diajukan.

```
Widget _ikonWarga({required BuildContext context, required bool terpilih}) {
  final total = context.select<PengajuanModel, int>(
    (model) => model.totalPengajuan,
  );

  return Badge(
    isLabelVisible: total > 0,
    label: Text(total.toString()),
    child: Icon(terpilih ? Icons.people : Icons.people_outline),
  ); // Badge
```

context.select() digunakan untuk mengambil nilai totalPengajuan dari model. Ketika jumlah pengajuan berubah, angka pada badge ikut diperbarui. Widget ikon Warga sebaiknya dipisahkan agar pembaruan badge tidak menyebabkan seluruh struktur navigasi dibangun ulang.

Badge tersebut diterapkan pada NavigationBar untuk layar sempit dan NavigationRail untuk layar lebar. Fitur navigasi lain, seperti named routes dan NavigationDrawer, tetap dipertahankan.

**Soal 2: Buat Kerangka Desain Tugas Akhir Saudara**

a. Membuat StatelessWidget

Membuat halaman Beranda dan Laporan menggunakan StetelessWidget. Widget ini digunakan untuk menampilkan tampilan yang tidak mengelola state lokal secara langsung. Halaman Beranda menampilkan informasi aplikasi dan jumlah laporan favorit, sedangkan halaman Laporan menampilkan daftar laporan warga.

```
class HalamanBeranda extends StatelessWidget {
  const HalamanBeranda({super.key});

  @override
  Widget build(BuildContext context) {
    final jumlahFavorit = context.select<FavoritModel, int>(
      (model) => model.daftarFavorit.length,
    );
```

b. Membuat StatefulWidget

Membuat halaman Pengaturan menggunakan StetefulWidget untuk mengelola status notifikasi. Perubahan nilai dilakukan menggunakan setState() agar tampilan diperbarui ketika pengguna mengaktifkan atau

menonaktifkan Switch.

```
SwitchListTile(
  secondary: Icon(
    notifikasiAktif
        ? Icons.notifications_active
        : Icons.notifications_off,
  ), // Icon
  title: const Text('Notifikasi'),
  subtitle: Text(
    notifikasiAktif
        ? 'Notifikasi laporan aktif'
        : 'Notifikasi laporan dinonaktifkan',
  ), // Text
  value: notifikasiAktif,
  onChanged: (value) {
    setState(() {
      notifikasiAktif = value;
    });
  },
), // SwitchListTile
```

c. Menerapkan Manajemen State dengan Provider Membuat model FavoritModel yang menggunakan ChangeNotifier untuk menyimpan daftar laporan favorit. Model tersebut dipasang melalui ChangeNotifierProvider di tingkat root aplikasi sehingga dapat diakses oleh beberapa halaman.

```
import 'package:flutter/foundation.dart';

class FavoritModel extends ChangeNotifier {
  final Set<String> _daftarFavorit = {};

  Set<String> get daftarFavorit => Set.unmodifiable(_daftarFavorit);

  bool isFavorit(String judul) {
    return _daftarFavorit.contains(judul);
  }

  void toggleFavorit(String judul) {
    if (_daftarFavorit.contains(judul)) {
      _daftarFavorit.remove(judul);
    } else {
      _daftarFavorit.add(judul);
    }

    notifyListeners();
  }
}
```

d. Menggabungkan Seluruh Komponen pada main.dart Menggabungkan seluruh halaman melalui MaterialApp, konfigurasi named routes, serta Provider pada tingkat root aplikasi. Setelah itu, aplikasi dijalankan untuk memastikan navigasi dan perubahan state berfungsi

dengan baik.

```dart
import 'package:flutter/material.dart';

import 'navigation/app_routes.dart';

import 'package:provider/provider.dart';

import 'models/favorit_model.dart';

Run | Debug | Profile
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => FavoritModel(),
      child: const AplikasiSahabatWarga(),
    ), // ChangeNotifierProvider
  );
}

class AplikasiSahabatWarga extends StatelessWidget {
  const AplikasiSahabatWarga({super.key});

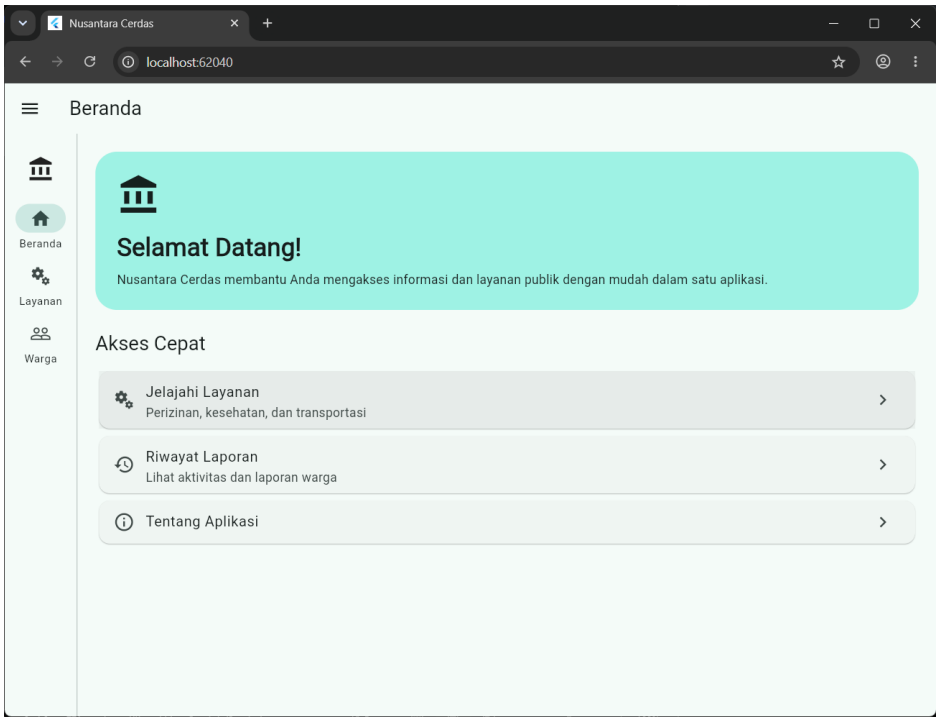
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Sahabat Warga',
      theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.indigo),
      initialRoute: AppRoutes.beranda,
      routes: AppRoutes.daftarRoute(),
      onGenerateRoute: AppRoutes.bentukRoute,
      onUnknownRoute: AppRoutes.routeTidakDikenal,
    ); // MaterialApp
  }
}

```

4. Hasil dan dokumentasi Program

**Soal 1: Aplikasi Layanan Warga Smart City "Nusantara Cerdas Mobile"**

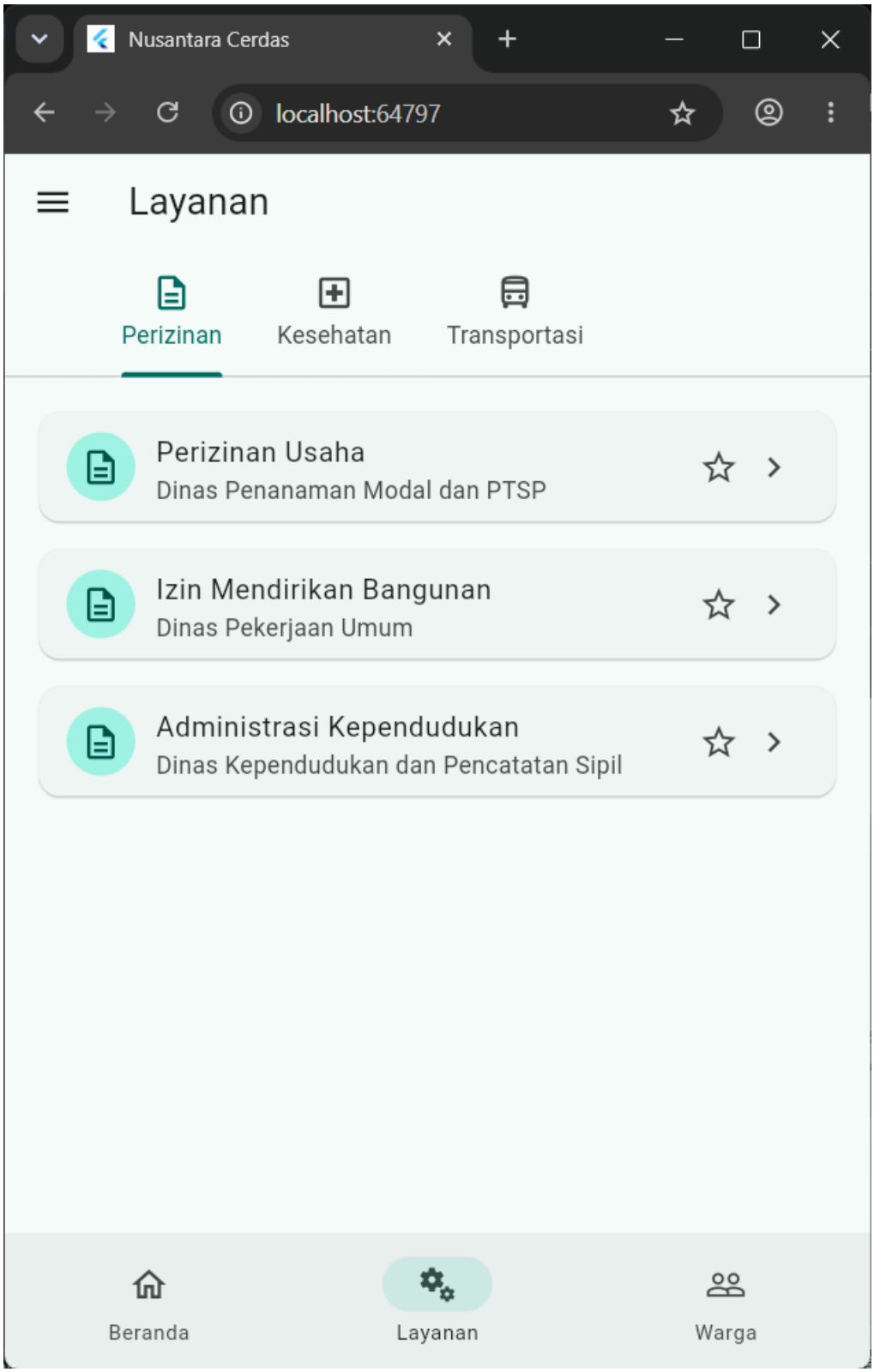
a. Tampilan Awal Aplikasi




Halaman awal aplikasi Nusantara Cerdas yang menampilkan informasi

singkat mengenai aplikasi dan beberapa akses cepat. Halaman ini menjadi tampilan awal sebelum pengguna memilih tujuan navigasi lain.

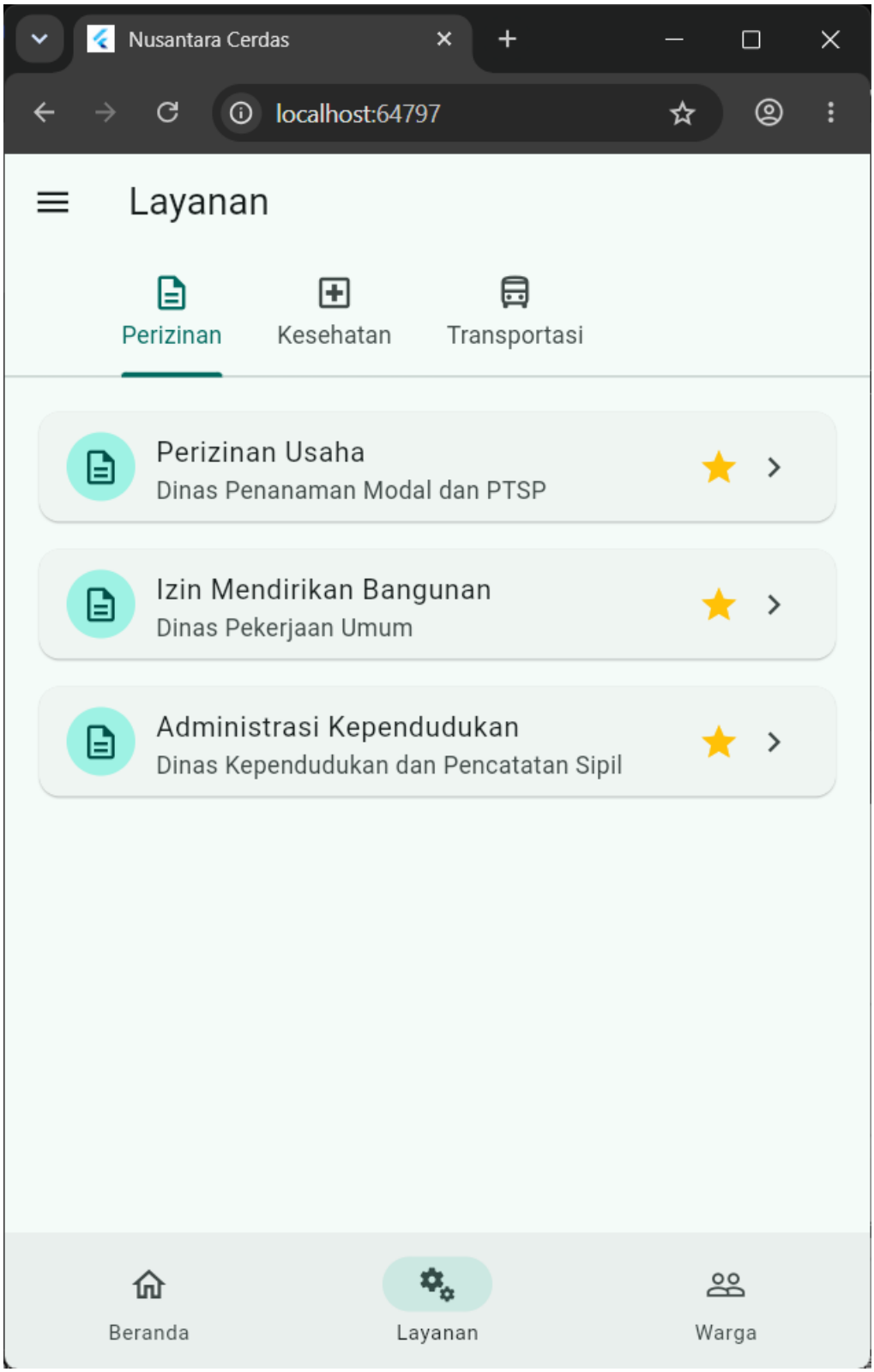
b. Tampilan Daftar Layanan




Menampilkan daftar layanan pemerintah yang tersedia dalam aplikasi.

Setiap layanan memiliki ikon bintang yang dapat digunakan untuk menandai layanan sebagai favorit.

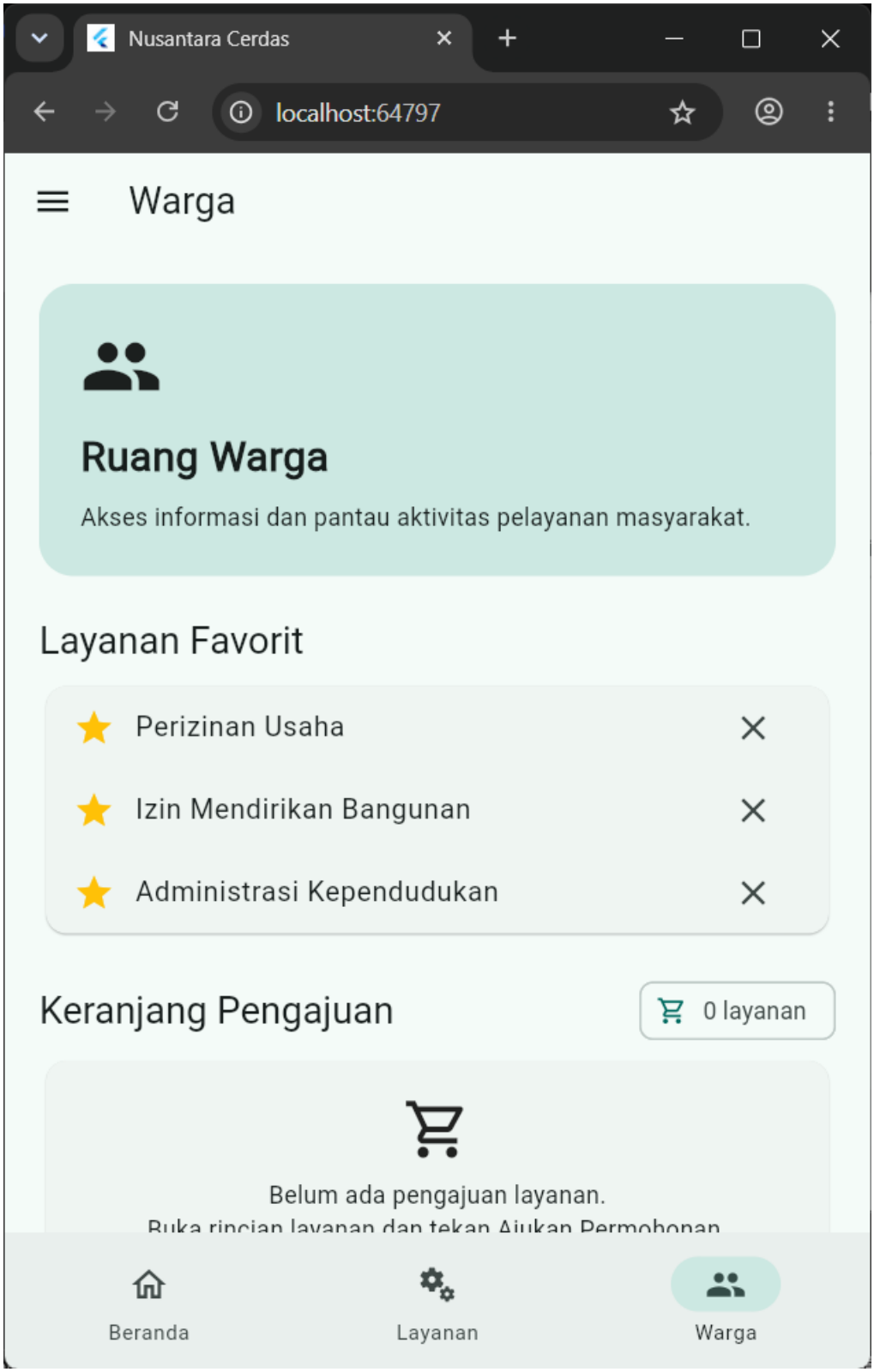
c. Tampilan Setelah Perubahan State Favorit




Menunjukkan perubahan tampilan ketika pengguna memilih layanan

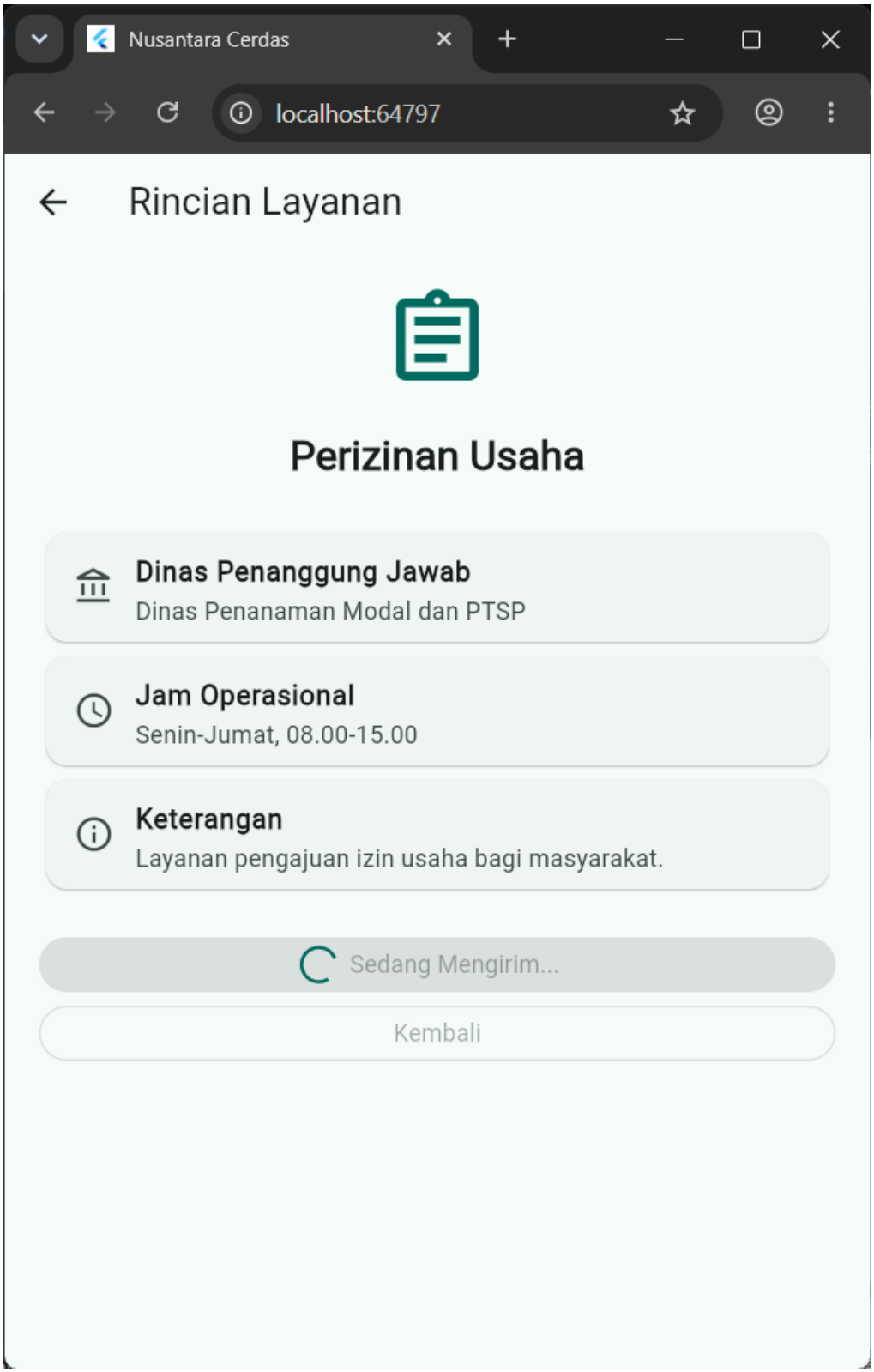
sebagai favorit. Ikon bintang berubah menjadi bintang warna kuning. Perubahan ini terjadi secara langsung karena FavoritModel memanggil notifyListeners() setelah data berubah.

d. Tampilan Halaman Warga



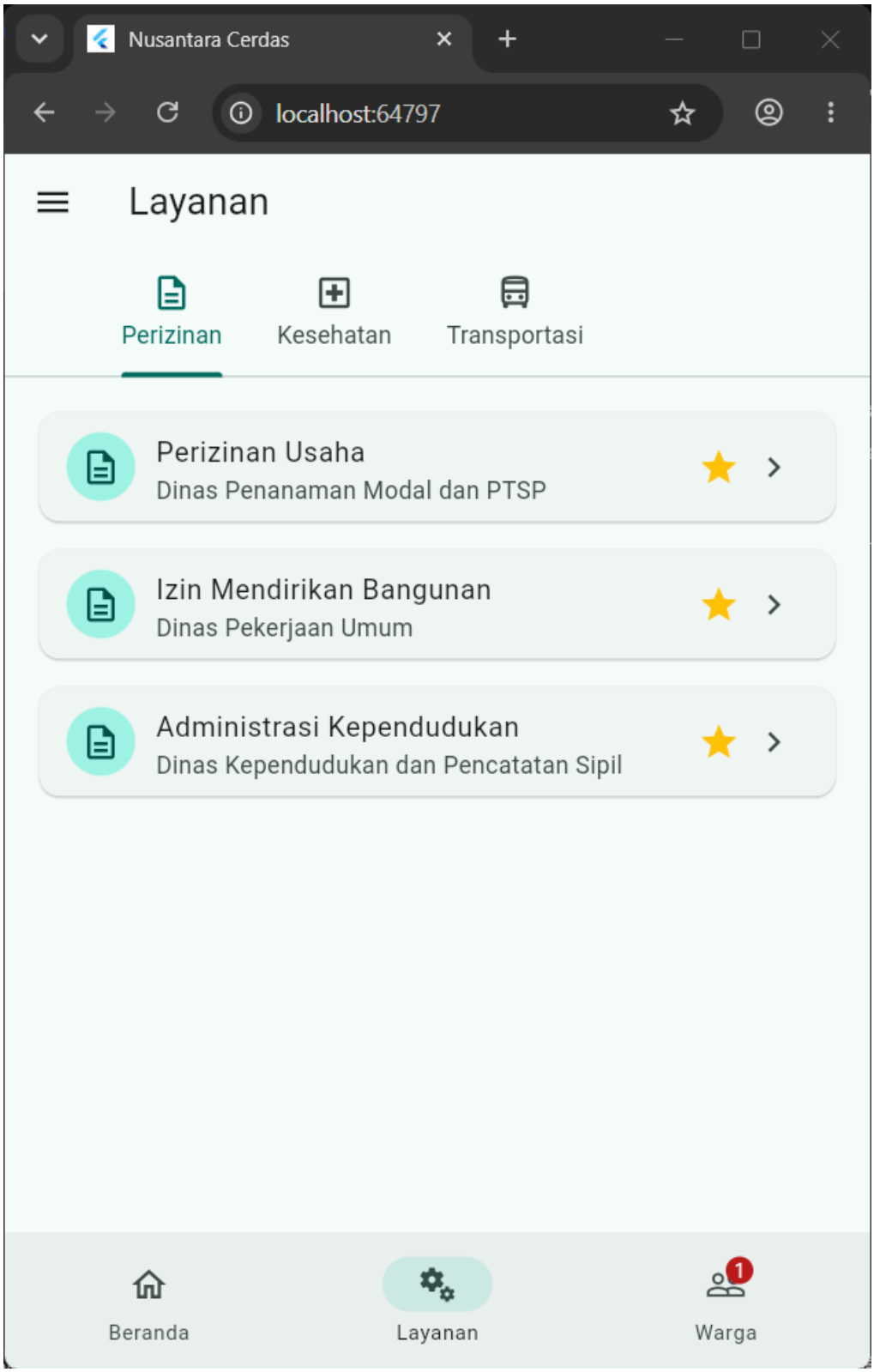
Halaman warga yang menampilkan layanan favorit dan layanan yang telah diajukan. Data tersebut berasal dari FavoritModel dan PengajuanModel. Penggunaan widget Consumer memungkinkan bagian tampilan terkait diperbarui ketika data model berubah.

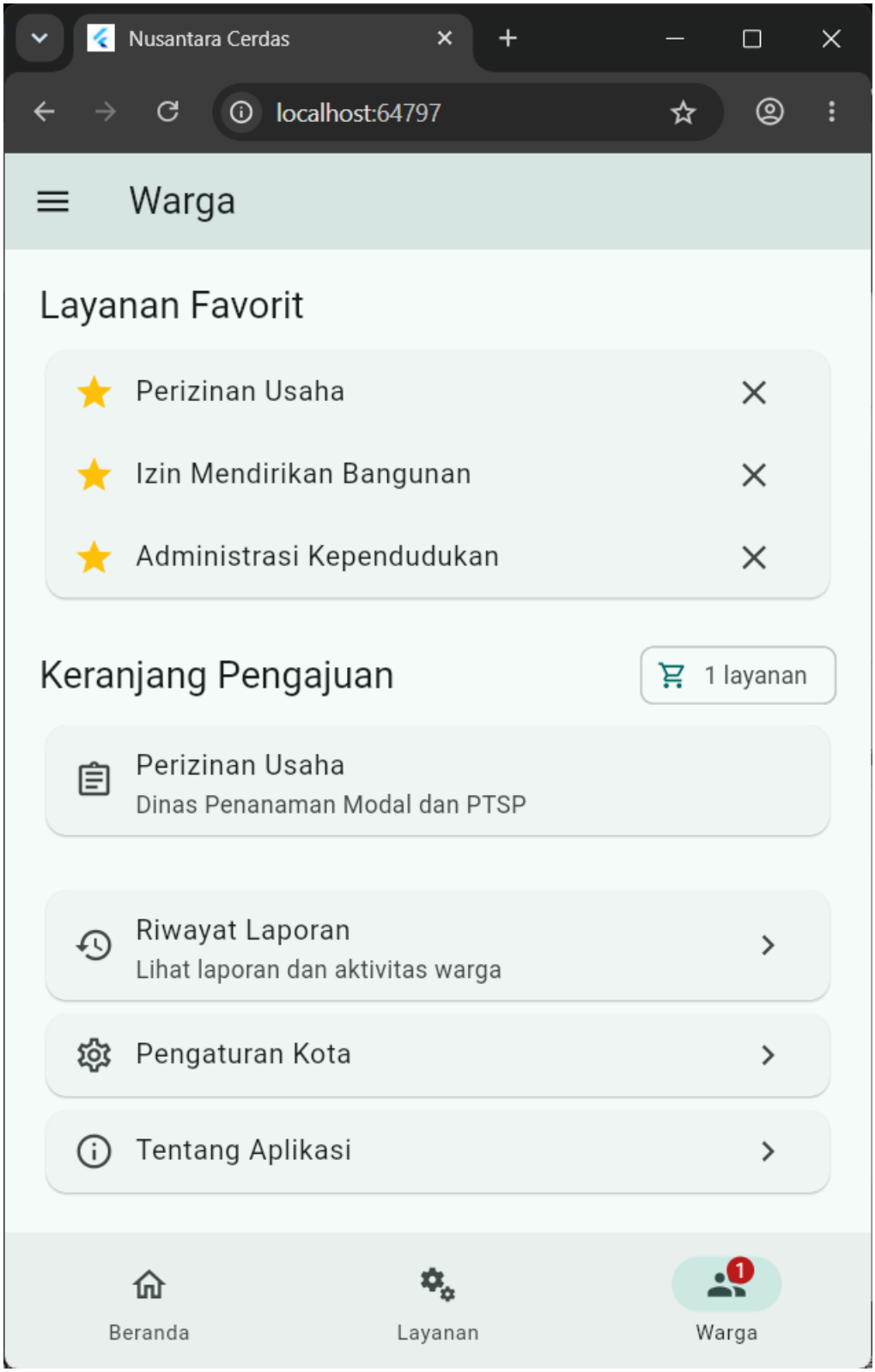
e. Tampilan Proses Pengajuan Layanan




Menunjukkan kondisi aplikasi ketika pengguna menekan tombol ”Ajukan Permohonan”. Tombol berubah menjadi status ”Sedang Mengirim...” dan menampilkan indikator loading.

f. Tampilan Setelah Pengajuan Berhasil

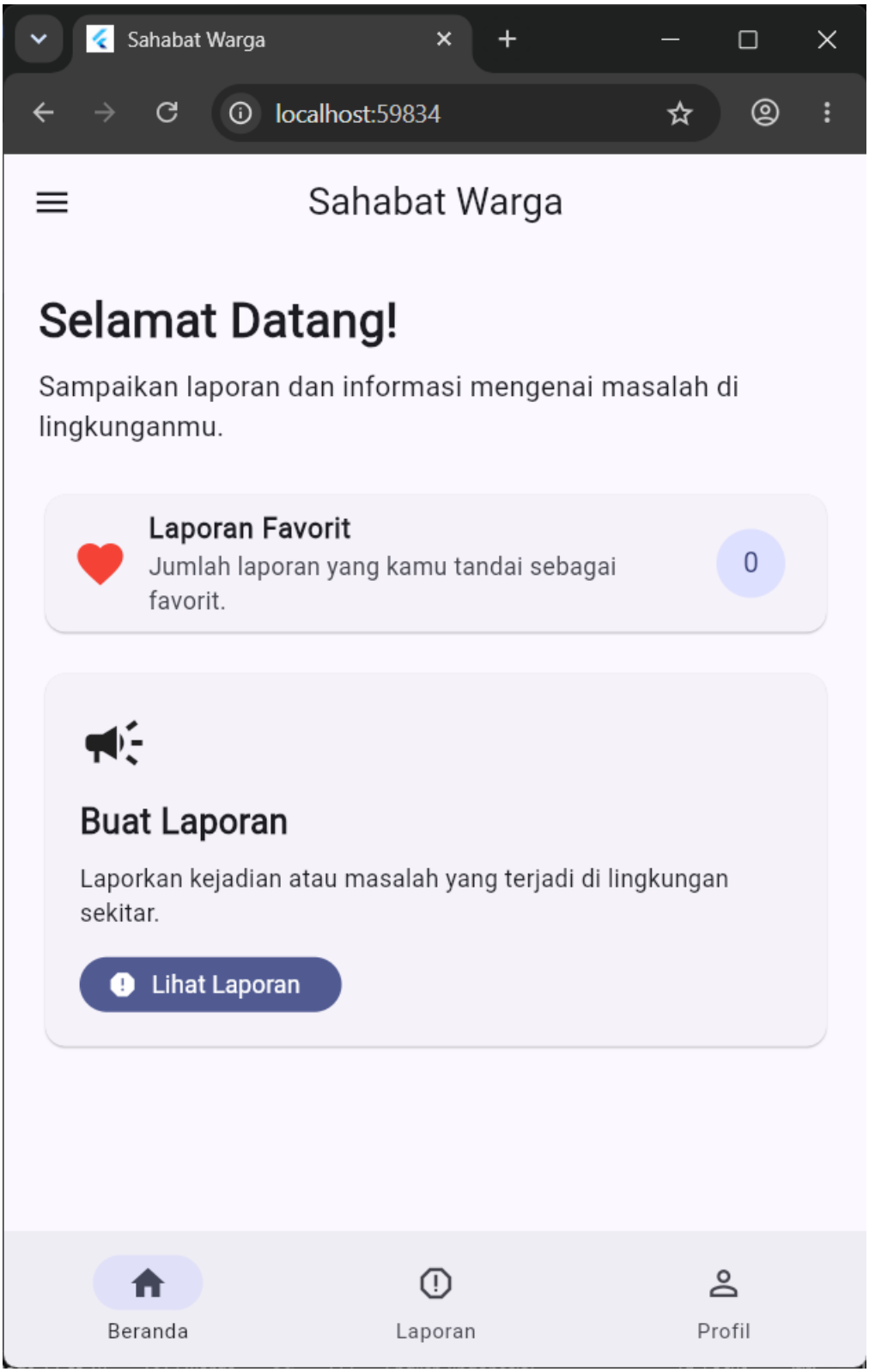




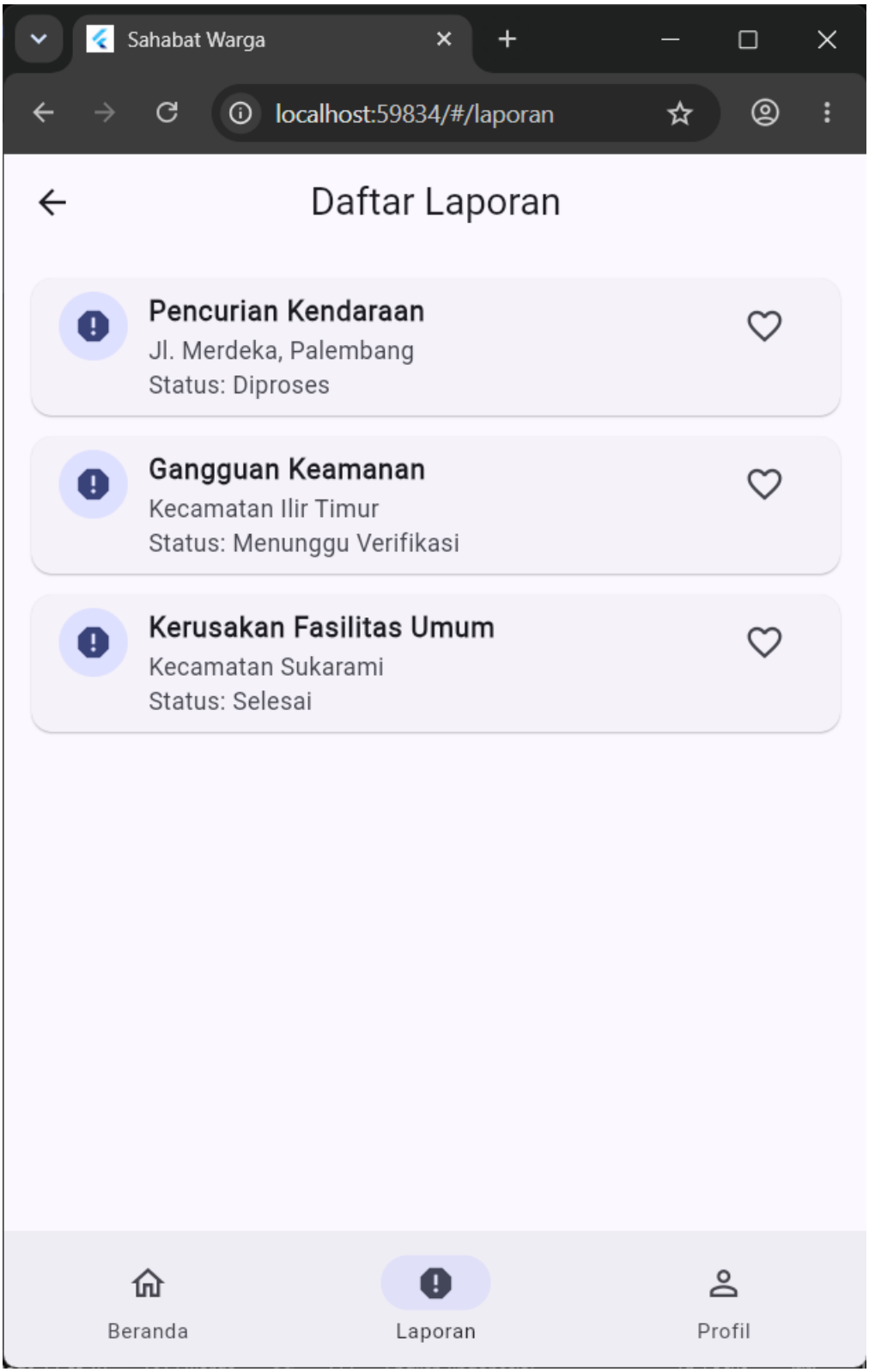

Memperlihatkan perubahan setelah layanan berhasil ditambahkan ke daftar pengajuan. Jumlah pengajuan pada halaman Warga bertambah dan badge pada ikon navigasi Warga menampilkan angka yang sesuai.

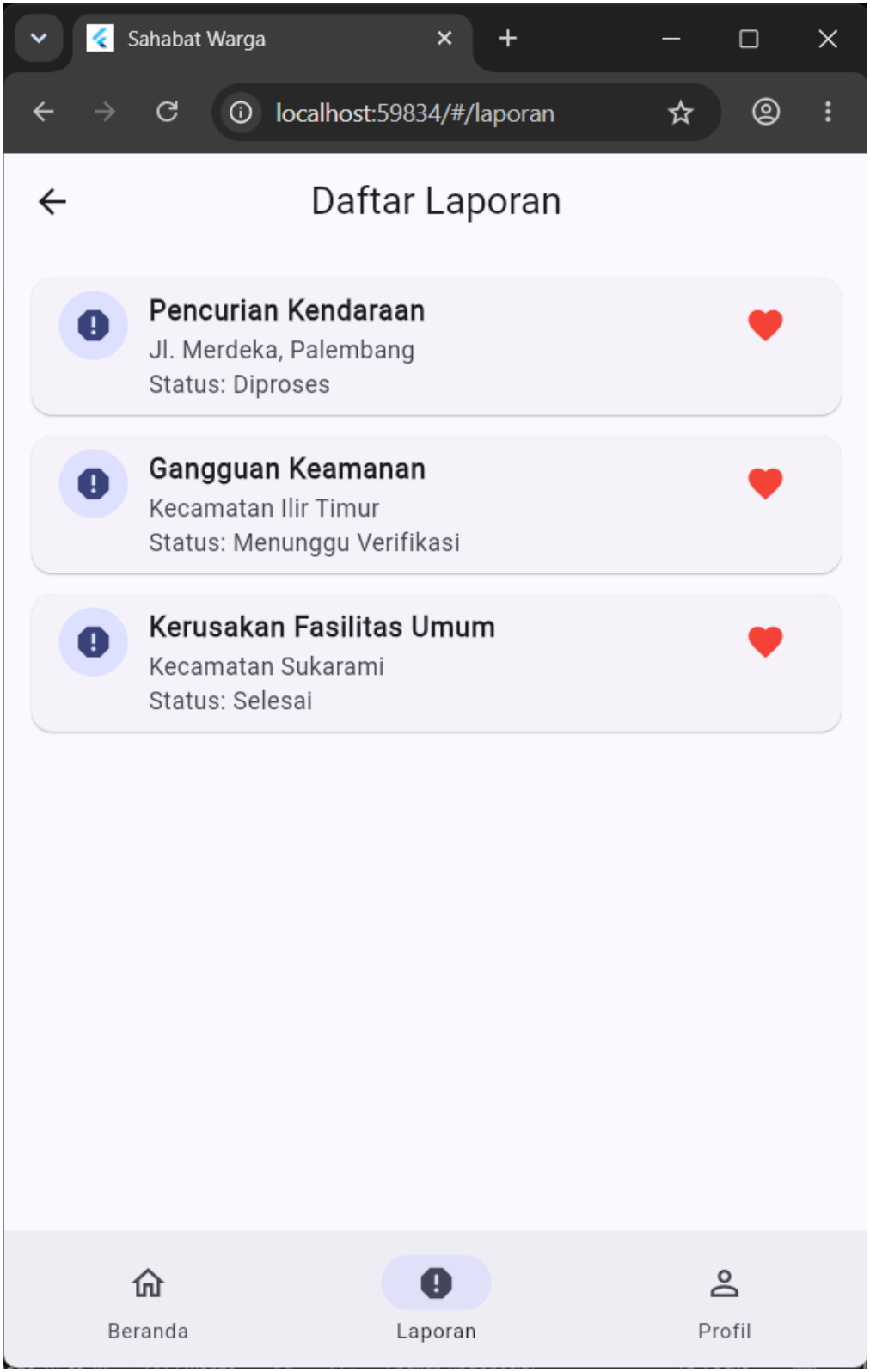
**Soal 2: Menambahkan Manajemen State pada Rancangan Tugas Akhir Saudara Ketentuan Pengerjaan**

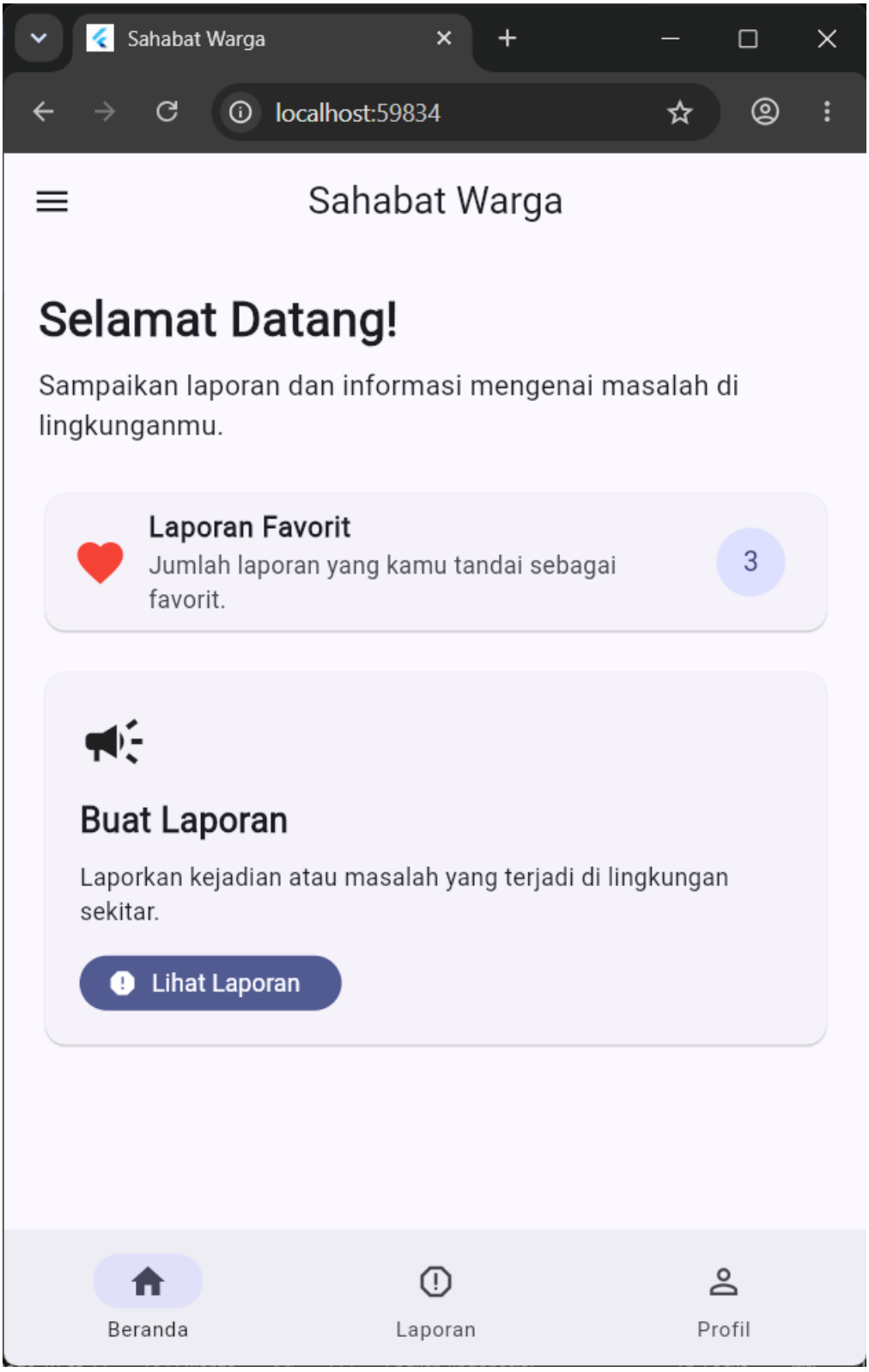
a. Tampilan Awal Aplikasi Sahabat Warga



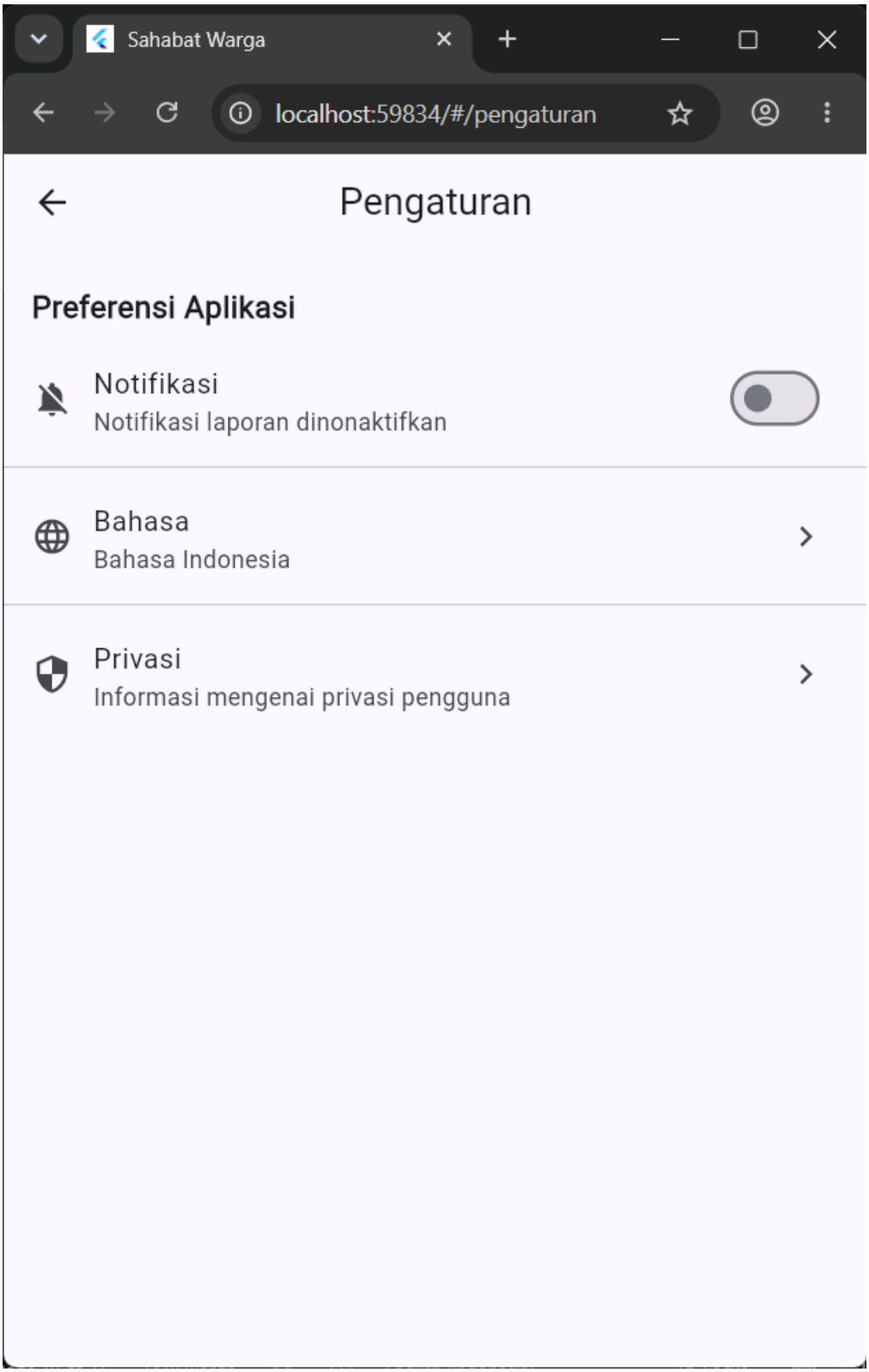
b. Tampilan Daftar Laporan Warga

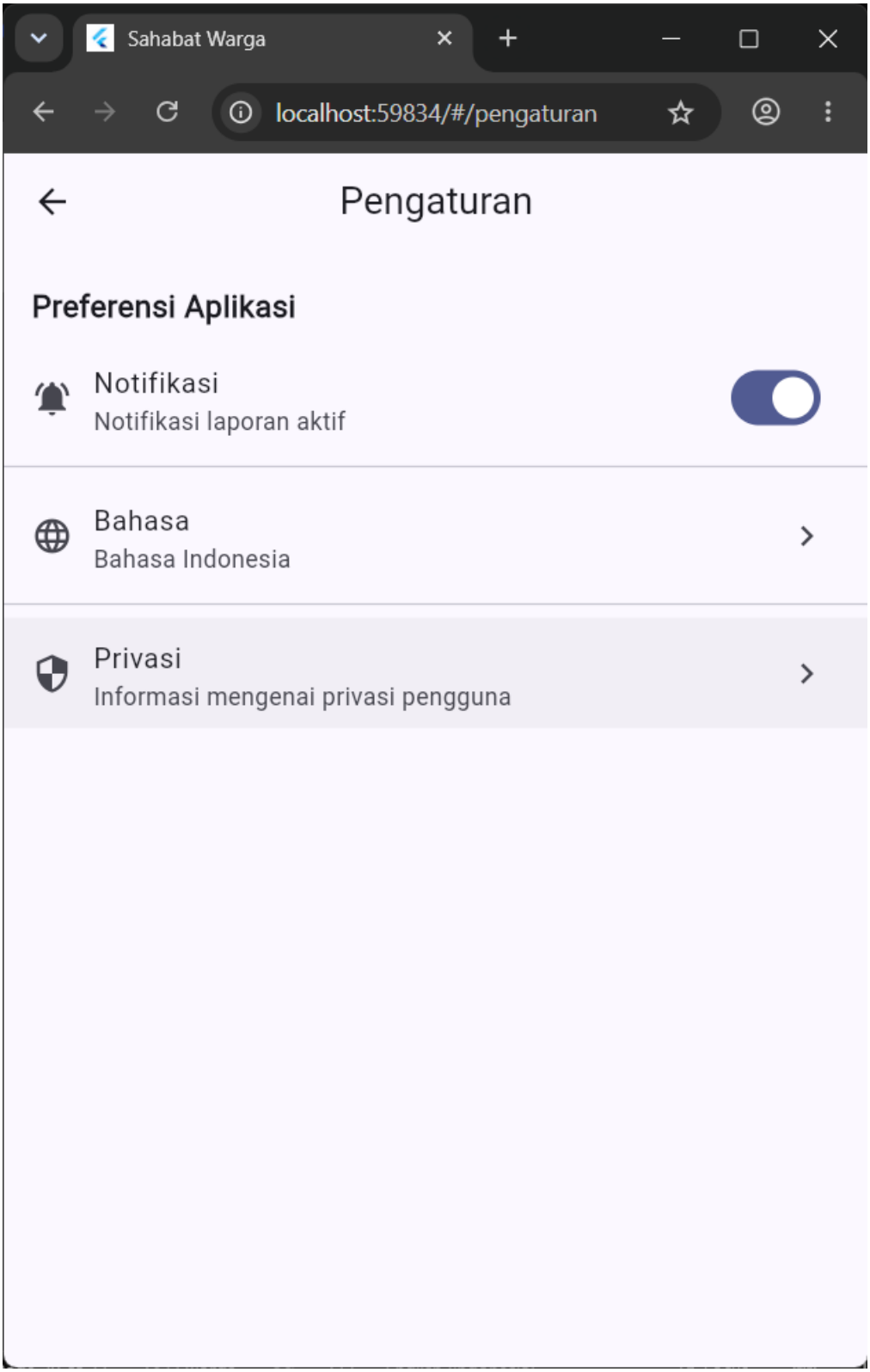



Sahabat Warga
localhost:59834/#/laporan
Daftar Laporan
Pencurian Kendaraan
Jl. Merdeka, Palembang
Status: Diproses
Gangguan Keamanan
Kecamatan Ilir Timur
Status: Menunggu Verifikasi
Kerusakan Fasilitas Umum
Kecamatan Sukarami
Status: Selesai
Beranda
Laporan
Profil


Sahabat Warga
localhost:59834
Sahabat Warga
Selamat Datang!
Sampaikan laporan dan informasi mengenai masalah di lingkunganmu.
Laporan Favorit
Jumlah laporan yang kamu tandai sebagai favorit.
3
Buat Laporan
Laporkan kejadian atau masalah yang terjadi di lingkungan sekitar.
Lihat Laporan
Beranda
Laporan
Profil

c. Tampilan Perubahan State pada Pengaturan






5. Analisis dan pembahasan
    a. Nusantara Cerdas

Aplikasi Nusantara Cerdas Mobile dikembangkan dengan menerapkan manajemen state menggunakan Provider dan ChangeNotifier. StatelessWidget digunakan pada bagian yang tidak memerlukan perubahan state lokal, seperti tampilan informasi dan komponen statis, sedangkan StatefulWidget digunakan pada halaman detail layanan untuk menampilkan indikator loading menggunakan setState().

Provider digunakan untuk mengelola data favorit dan pengajuan layanan agar perubahan dapat ditampilkan secara otomatis pada beberapa halaman. Kendala yang dapat ditemukan meliputi package Provider yang belum terpasang, kesalahan import model, atau tampilan yang tidak diperbarui. Kendala tersebut dapat diatasi dengan memasang dependensi Provider, memeriksa struktur dan import file, serta memastikan notifyListeners() dipanggil ketika data berubah.

Aplikasi ini berkaitan dengan pilar Smart Governance karena menyediakan akses informasi dan layanan publik secara digital sehingga dapat mempermudah interaksi masyarakat dengan pemerintah.

b. Sahabat Warga

Aplikasi Sahabat Warga menggunakan StatelessWidget pada halaman yang berfokus pada penyajian tampilan dan StatefulWidget pada halaman Pengaturan karena status notifikasi perlu berubah sesuai interaksi pengguna. Manajemen state bersama diterapkan menggunakan Provider agar data favorit dapat digunakan pada halaman Laporan dan Beranda secara konsisten.

Kendala yang mungkin muncul adalah kesalahan import, konfigurasi named routes, atau ProviderNotFoundException; hal tersebut dapat diatasi dengan memeriksa path import, memastikan rute terdaftar dengan benar, serta menempatkan Provider di atas MaterialApp.

Aplikasi ini berkaitan dengan pilar Smart Governance karena mendukung penyampaian informasi dan laporan warga melalui aplikasi digital sehingga dapat membantu meningkatkan partisipasi masyarakat dalam menyampaikan masalah di lingkungan sekitar.

6. Kesimpulan

a. Nusantara Cerdas

menerapkan manajemen state pada aplikasi Nusantara Cerdas Mobile menggunakan Provider, ChangeNotifier, Consumer, dan setState(). Melalui penerapan tersebut, data favorit dan pengajuan layanan dapat dikelola serta ditampilkan secara reaktif, sedangkan indikator loading dan badge membantu memberikan informasi kepada pengguna. Praktikum ini juga meningkatkan pemahaman mengenai perbedaan StatelessWidget dan StatefulWidget serta

penerapan navigasi responsif. Aplikasi yang dikembangkan diharapkan dapat mendukung konsep Smart Governance melalui penyediaan layanan publik yang lebih mudah diakses secara digital.

b. Sahabat Warga

Aplikasi Flutter Sahabat Warga yang menerapkan layout, navigasi, dan manajemen state. Penggunaan StatelessWidget, StatefulWidget, setState(), serta Provider membantu memahami perbedaan pengelolaan tampilan dan data yang digunakan bersama oleh beberapa halaman. Melalui praktikum ini, mahasiswa juga memahami penerapan named routes dan pengiriman data ant halaman sebagai dasar pengembangan aplikasi yang mendukung konsep Smart City.

7. Lampiran berisi Link Repositori GitHub

https://github.com/Jajoo06/PPBL_Modul2